

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Cache Memory

- Cache memory
 - The level of the memory hierarchy closest to the CPU

Given accesses $X_1, \dots, X_{n-1}, X_n$

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

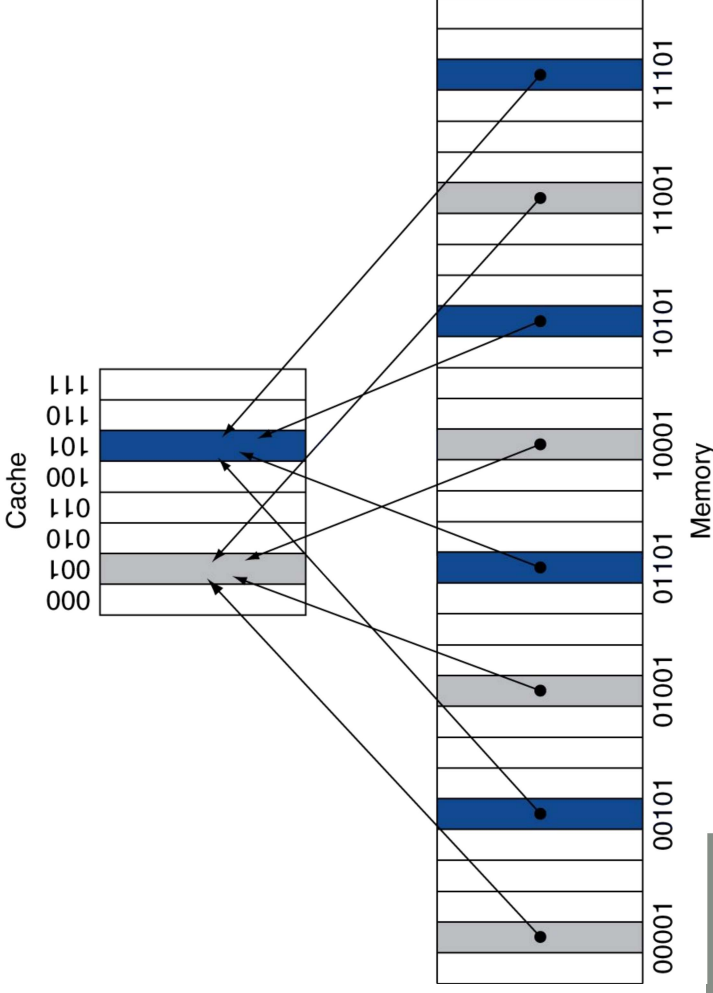
X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

a. Before the reference to X_n b. After the reference to X_n

- How do we know if the data is present?
- Where do we look?

Direct Mapped Cache

- Location determined by address
- Direct mapped: only one choice
 - (Block address) modulo (#Blocks in cache)



- #Blocks is a power of 2
- Use low-order address bits

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
 - Store block address as well as the data
 - Actually, only need the high-order bits
 - Called the tag
- What if there is no data in a location?
 - Valid bit: 1 = present, 0 = not present
 - Initially 0

Cache Example

- 8-blocks, 1 word/block, direct mapped
- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

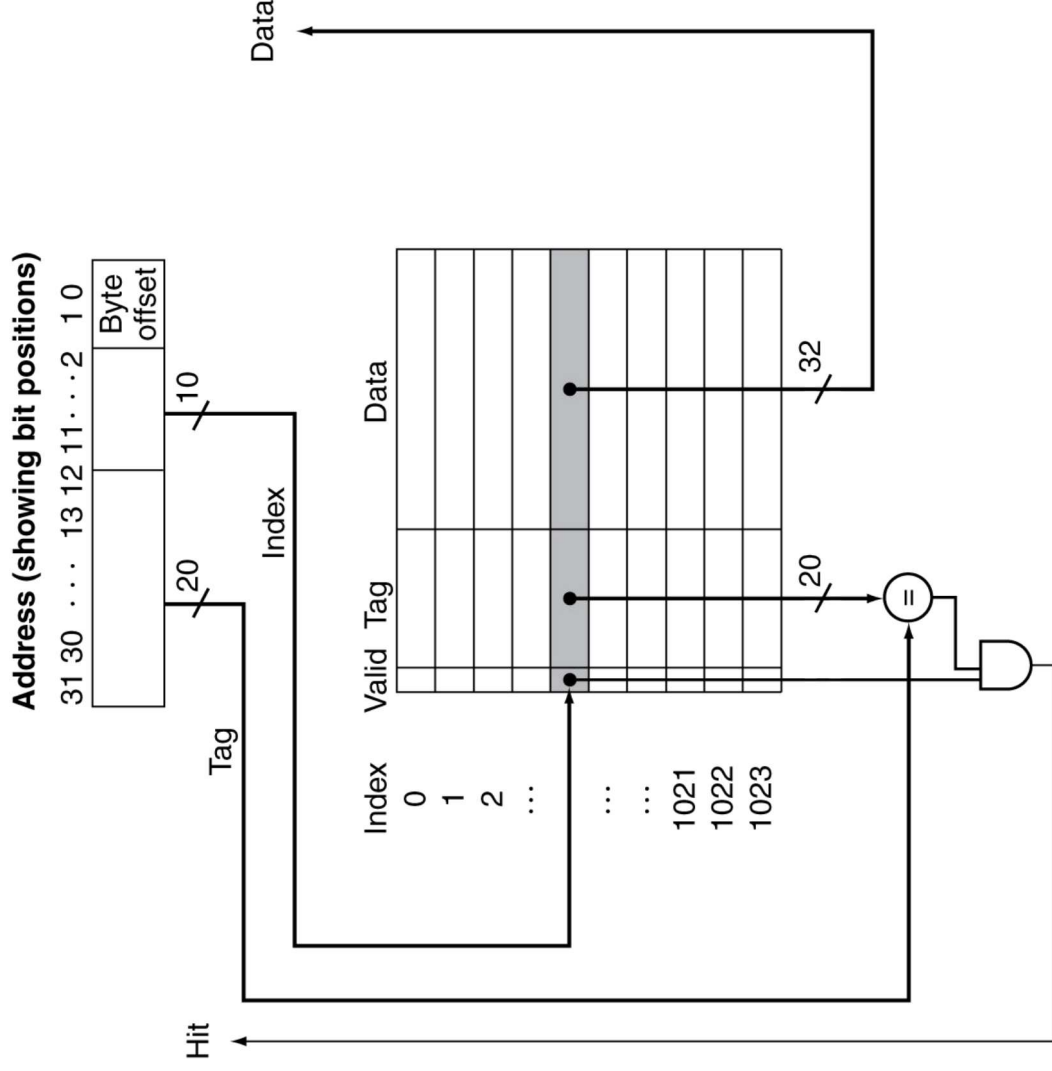
Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	11	Mem[11010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

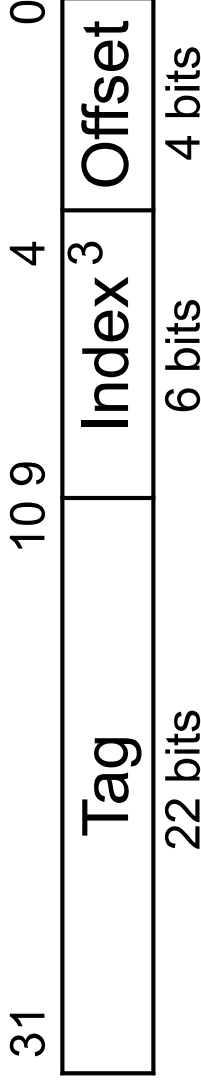
Index	V	Tag	Data
000	Y	10	Mem[100000]
001	N		
010	Y	10	Mem[10010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Address Subdivision



Example: Larger Block Size

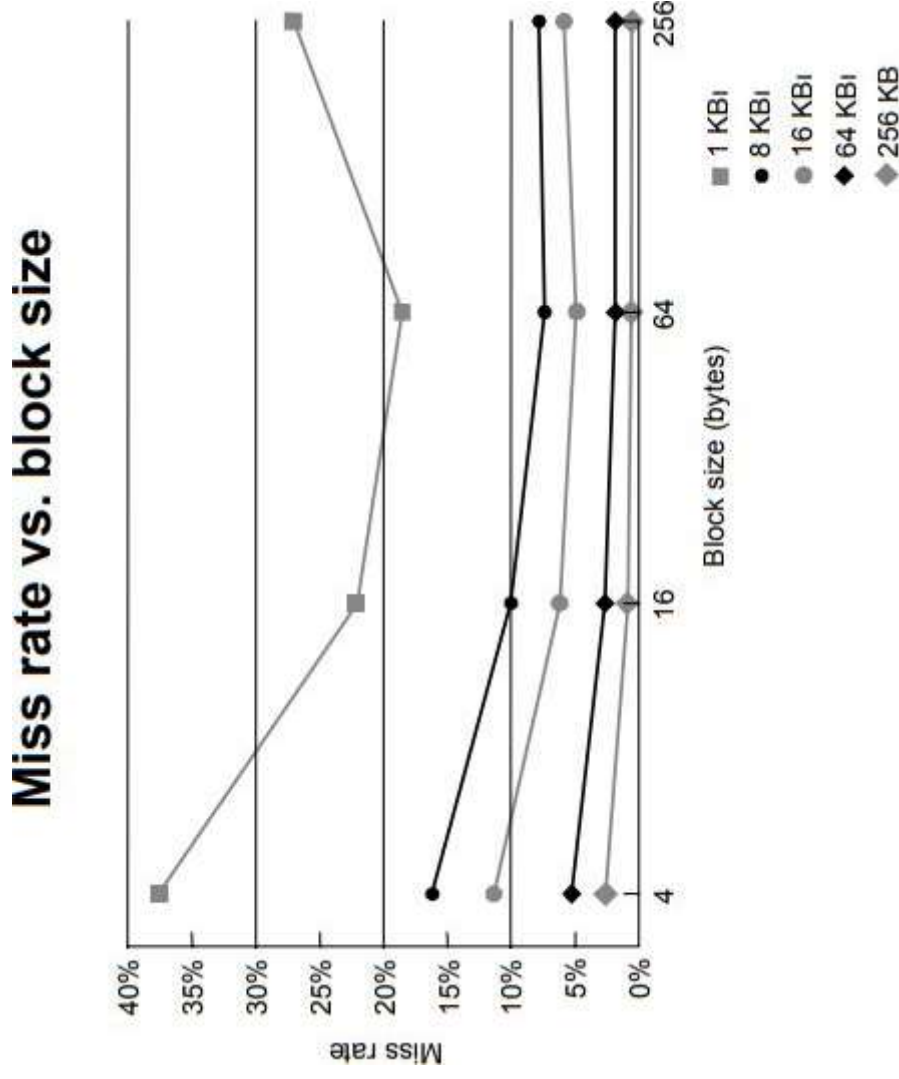
- 64 blocks, 16 bytes/block
 - To what block number does address 1200 map?
- Block address = $\lceil 1200/16 \rceil = 75$
- Block number = $75 \text{ modulo } 64 = 11$



Block Size Considerations

- Larger blocks (**size**) should reduce miss rate
 - Due to spatial locality
- But in a fixed-sized cache
 - Larger size blocks $\Rightarrow$ fewer of them
 - More competition $\Rightarrow$ increased miss rate
 - Larger size blocks $\Rightarrow$ pollution
- Larger miss penalty
 - Can override benefit of reduced miss rate
 - Early restart and critical-word-first can help

Block Size Considerations



Cache Misses

- On cache hit, CPU proceeds normally
- On cache miss
 - Stall the CPU pipeline
 - Fetch block from next level of hierarchy
 - Instruction cache miss
 - Restart instruction fetch
 - Data cache miss
 - Complete data access

Write-Through

- On data-write hit, could just update the block in cache
 - But then cache and memory would be inconsistent
- Write through: also update memory
- But makes writes take longer
 - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
 - Effective CPI = $1 + 0.1 \times 100 = 11$
- Solution: write buffer
 - Holds data waiting to be written to memory
 - CPU continues immediately
 - Only stalls on write if write buffer is already full

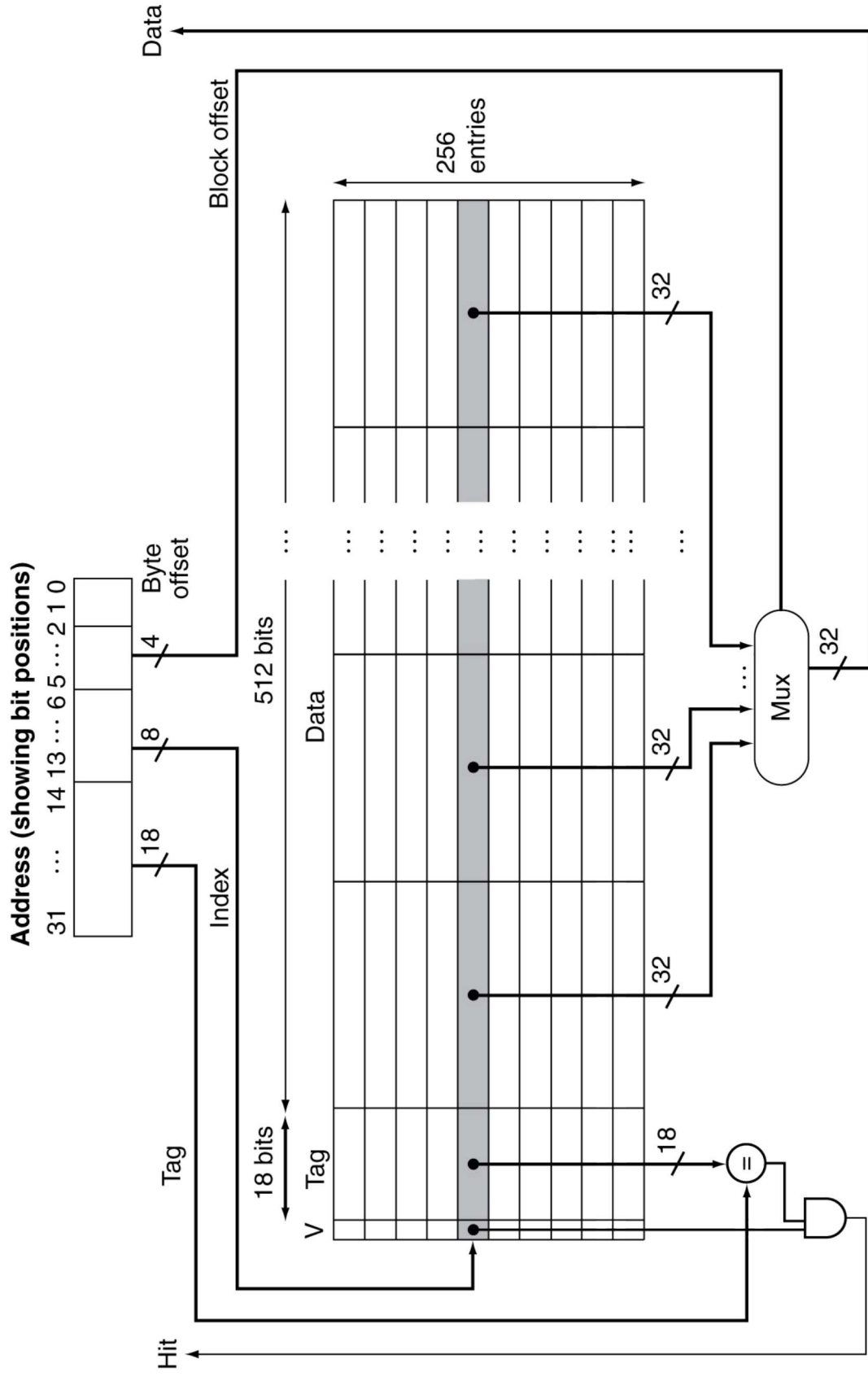
Write-Back

- Alternative: On data-write hit, just update the block in cache
 - Keep track of whether each block is dirty
- When a dirty block is replaced
 - Write it back to memory
 - Can use a write buffer to allow replacing block to be read first

Example: Intrinsicity FastMATH

- Embedded MIPS processor
 - 12-stage pipeline
 - Instruction and data access on each cycle
- Split cache: separate I-cache and D-cache
 - Each 16KB: 256 blocks × 16 words/block
 - D-cache: write-through or write-back
- SPEC2000 miss rates
 - I-cache: 0.4%
 - D-cache: 11.4%
 - Weighted average: 3.2%

Example: Intrinsicity FastMATH



Main Memory Supporting Caches

- Use DRAMs for main memory
 - Fixed width (e.g., 1 word)
 - Connected by fixed-width clocked bus
 - Bus clock is typically slower than CPU clock
- Example cache block read
 - 1 bus cycle for address transfer
 - 15 bus cycles per DRAM access
 - 1 bus cycle per data transfer
- For 4-word block, 1-word-wide DRAM
 - Miss penalty = $1 + 4 \times 15 + 4 \times 1 = 65$ bus cycles
 - Bandwidth = $16 \text{ bytes} / 65 \text{ cycles} = 0.25 \text{ B/cycle}$

Measuring Cache Performance

- Components of CPU time
 - Program execution cycles
 - Includes cache hit time
 - Memory stall cycles
 - Mainly from cache misses
- With simplifying assumptions:

Memory stall cycles

$$= \frac{\text{Memory accesses} \times \text{Miss rate} \times \text{Miss penalty}}{\text{Program}}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Cache Performance Example

- Given
 - I-cache miss rate = 2%
 - D-cache miss rate = 4%
 - Miss penalty = 100 cycles
 - Base CPI (ideal cache) = 2
 - Load & stores are 36% of instructions
- Miss cycles per instruction
 - I-cache: $0.02 \times 100 = 2$
 - D-cache: $0.36 \times 0.04 \times 100 = 1.44$
- Actual CPI = $2 + 2 + 1.44 = 5.44$
 - Ideal CPU is $5.44/2 = 2.72$ times faster

Average Access Time

- Hit time is also important for performance
- Average memory access time (AMAT)
 - $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
- Example
 - CPU with 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, l-cache miss rate = 5%
 - $AMAT = 1 + 0.05 \times 20 = 2ns$
 - 2 cycles per instruction

Performance Summary

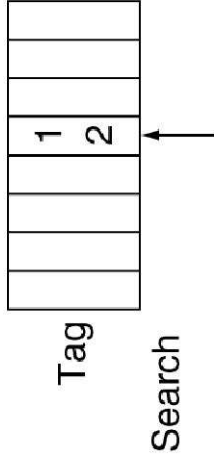
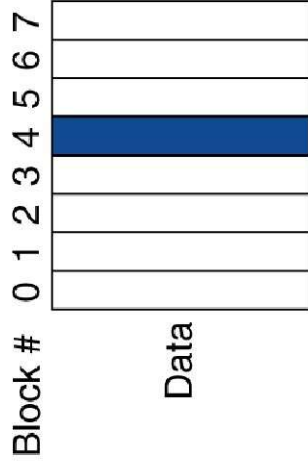
- When CPU performance increased
 - Miss penalty becomes more significant
- Decreasing base CPI
 - Greater proportion of time spent on memory stalls
- Increasing clock rate
 - Memory stalls account for more CPU cycles
- Can't neglect cache behavior when evaluating system performance

Associative Caches

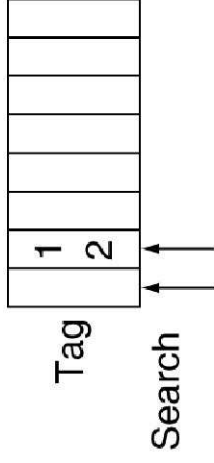
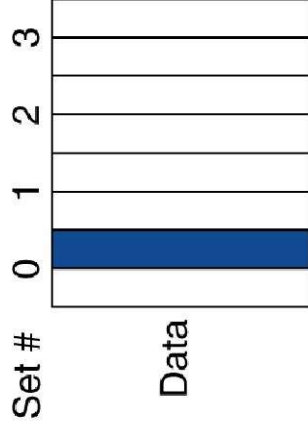
- Fully associative
 - Allow a given block to go in any cache entry
 - Requires all entries to be searched at once
 - Comparator per entry (expensive)
- n -way set associative
 - Each set contains n entries
 - Block number determines which set
 - (Block number) modulo (#Sets in cache)
 - Search all entries in a given set at once
 - n comparators (less expensive)

Associative Cache Example

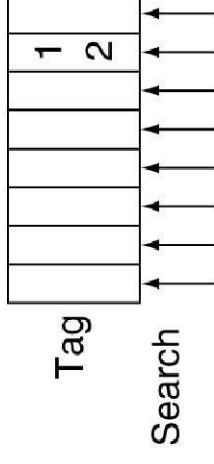
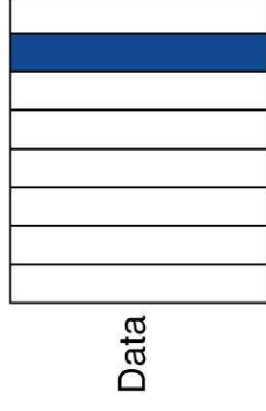
Direct mapped



Set associative



Fully associative



Spectrum of Associativity

- For a cache with 8 entries

One-way set associative
(direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example

- Compare 4-block caches
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Associativity Example

- 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access	
			Set 0	Set 1
0	0	miss	Mem[0]	
8	0	miss	Mem[0]	Mem[8]
0	0	hit	Mem[0]	Mem[8]
6	0	miss	Mem[0]	Mem[6]
8	0	miss	Mem[8]	Mem[6]

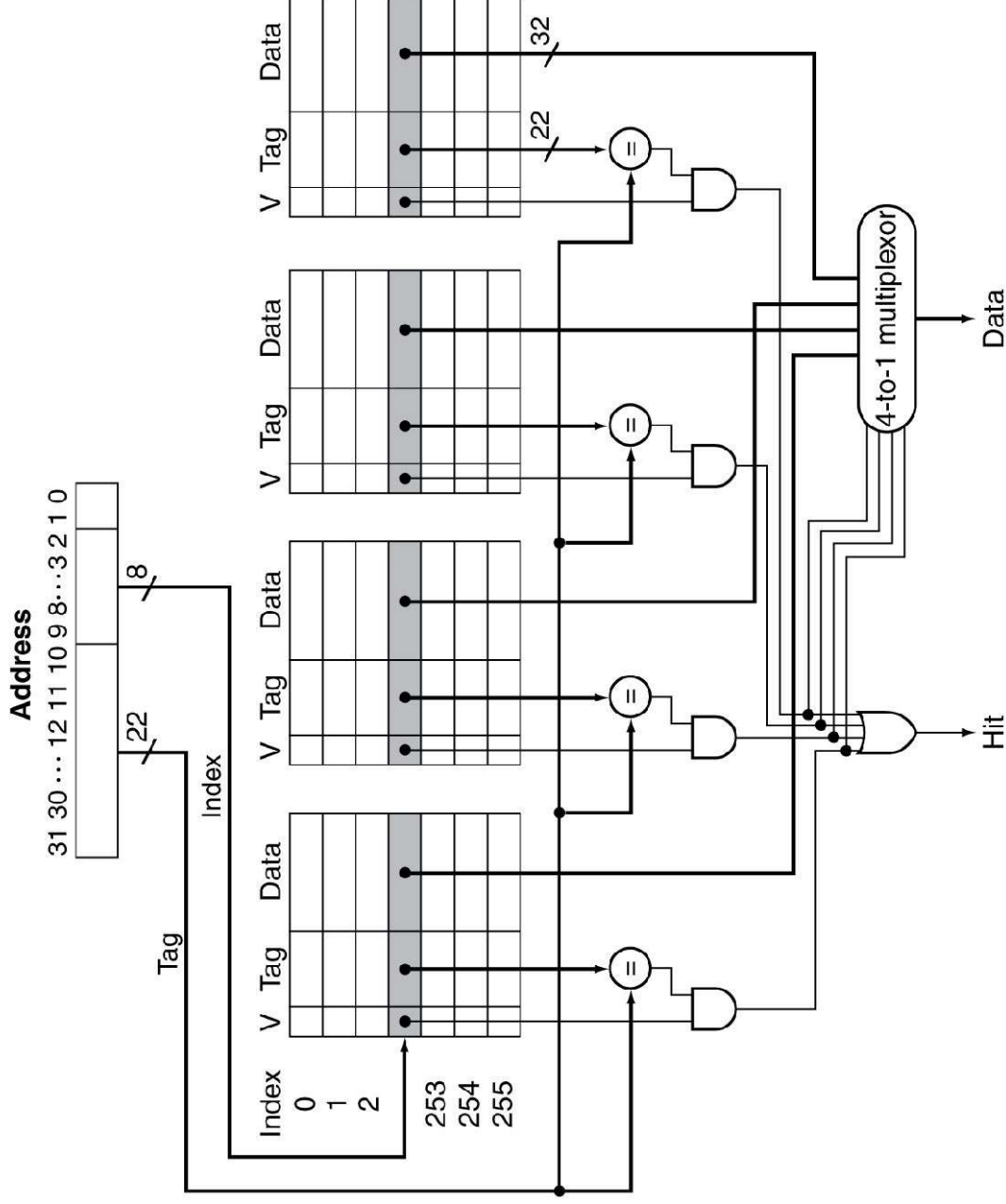
- Fully associative

Block address	Hit/miss	Cache content after access	
0	miss	Mem[0]	
8	miss	Mem[0]	Mem[8]
0	hit	Mem[0]	Mem[8]
6	miss	Mem[0]	Mem[8]
8	hit	Mem[0]	Mem[6]

How Much Associativity

- Increased associativity decreases miss rate
 - But with diminishing returns
- Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000
 - 1-way: 10.3%
 - 2-way: 8.6%
 - 4-way: 8.3%
 - 8-way: 8.1%

Set Associative Cache Organization



Replacement Policy

- Direct mapped: no choice
- Set associative
 - Prefer non-valid entry, if there is one
 - Otherwise, choose among entries in the set
- Least-recently used (LRU)
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
- Random
 - Gives approximately the same performance as LRU for high associativity

Multilevel Caches

- Primary cache attached to CPU
 - Small, but fast
- Level-2 cache services misses from primary cache
 - Larger, slower, but still faster than main memory
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache

Multilevel Cache Example

■ Given

- CPU base CPI = 1, clock rate = 4GHz
- Miss rate/instruction = 2%
- Main memory access time = 100ns

■ With just primary cache

- Miss penalty = $100\text{ns}/0.25\text{ns} = 400$ cycles
- Effective CPI = $1 + 0.02 \times 400 = 9$

Example (cont.)

- Now add L-2 cache
 - Access time = 5ns
 - Global miss rate to main memory = 0.5%
- Primary miss with L-2 hit
 - Penalty = $5\text{ns}/0.25\text{ns} = 20$ cycles
- Primary miss with L-2 miss
 - Extra penalty = 500 cycles
- $\text{CPI} = 1 + 0.02 \times 20 + 0.005 \times 400 = 3.4$
- Performance ratio = $9/3.4 = 2.6$

Multilevel Cache Considerations

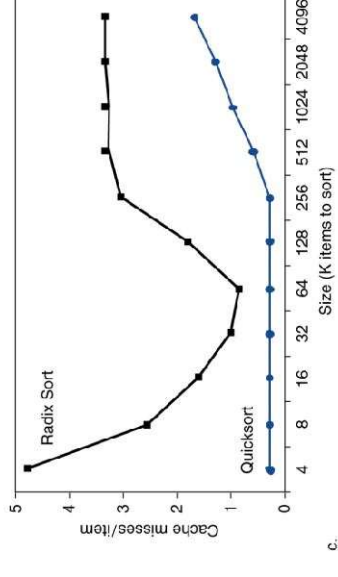
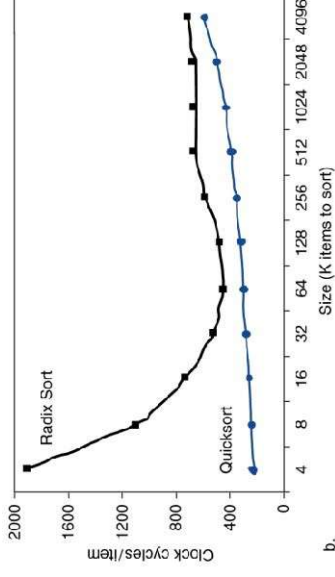
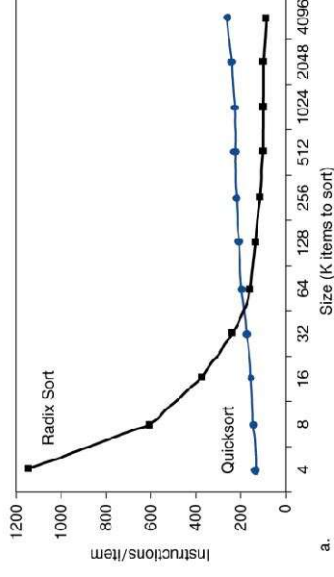
- Primary cache
 - Focus on minimal hit time
- L-2 cache
 - Focus on low miss rate to avoid main memory access
 - Hit time has less overall impact
- Results
 - L-1 cache usually smaller than a single cache
 - L-1 block size smaller than L-2 block size

Interactions with Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
 - Pending store stays in load/store unit
 - Dependent instructions wait in reservation stations
 - Independent instructions continue
- Effect of miss depends on program data flow
 - Much harder to analyse
 - Use system simulation

Interactions with Software

- Misses depend on memory access patterns
 - Algorithm behavior
 - Compiler optimization for memory access



Address Translation

Virtual Addresses:

Compiler generated programs on disk

Location of & references to code and data

It is used by CPU when a program accesses memory

VA addresses allow processes to run in isolated, protected memory spaces

Physical addresses in main memory:

Map virtual into physical addresses in actual locations in RAM

Memory Management Unit (MMU) translate virtual addresses into physical addresses

Multiple virtual addresses can map to same physical addresses (shared memory) or different locations (process isolation)

Page and Page Table

Page Table:

The **page table** is a data structure used by the operating system to map **virtual pages** to **physical frames**.

Memory is divided into fixed-size chunks:

Virtual memory → pages

Physical memory → frames

Each process has its own page table.

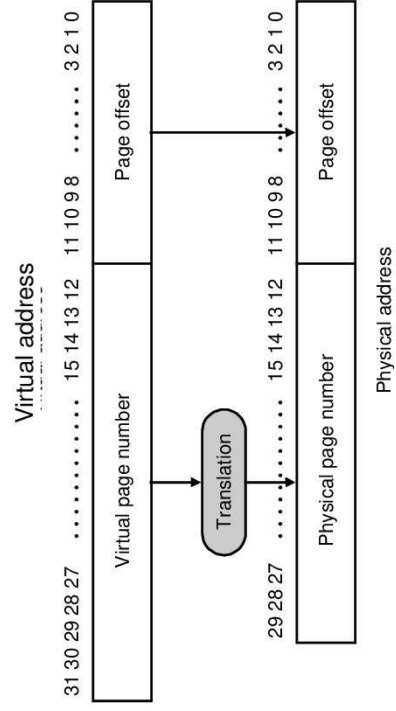
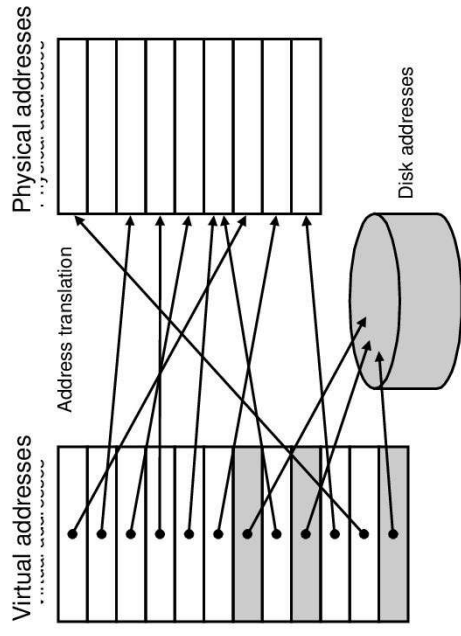
Example:

Virtual Page Number (VPN): 0x02

Physical Frame Number (PFN): 0x1A

⇒ VA 0x0200 → PA 0x1A00

Virtual Memory



Virtual memory addressing: page tables

Page table

- Library analogy: the card catalog
- Indexing all pages in virtual memory
- Each program has its own page table
- OS creates disk space for all the pages of every process
- The state of a process is specified by the page table, PC and registers
- Active and inactive processes (saving and loading page table register, PC and the other registers)

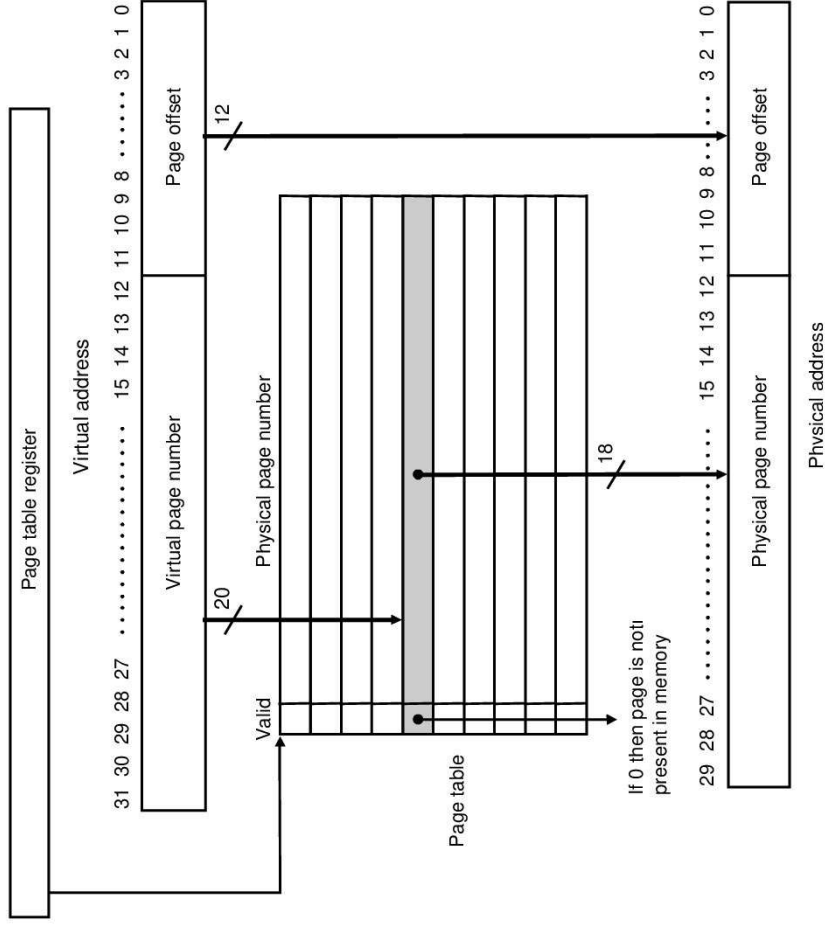
Page table entries

- Physical page or disk address
- Tag is not necessary (the index specifies the page address)
- Valid bit: 1 - the page is in memory; 0 - the page is on disk
- Reference (use) bit: LRU scheme for replacing pages
- Dirty bit (the page has been written): write-back scheme
- Information for the process using the page (memory protection)

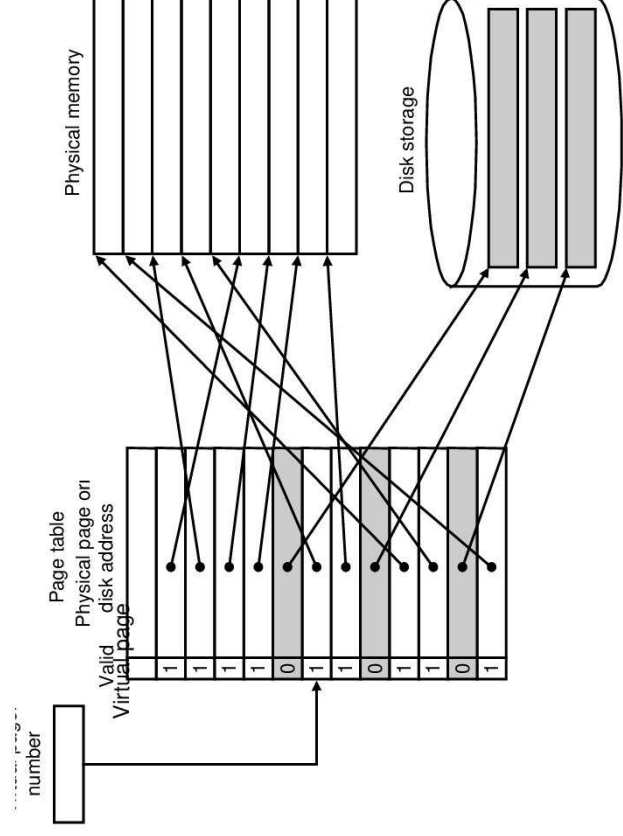
Example

- 32-bit virtual address, 4KB page, 4 byte entry - 4MB for a page table
- 250 processes X 4MB = 1GB !
- Reducing the page table size

Virtual memory addressing: address translation



Handling page faults



- Transfer the control to OS by using the exception mechanism
- Read the page from disk
- Determine which page in memory to replace (LRU)
- Write back to disk the replaced page (if its dirty bit is on)
- Write the physical address in the page table entry

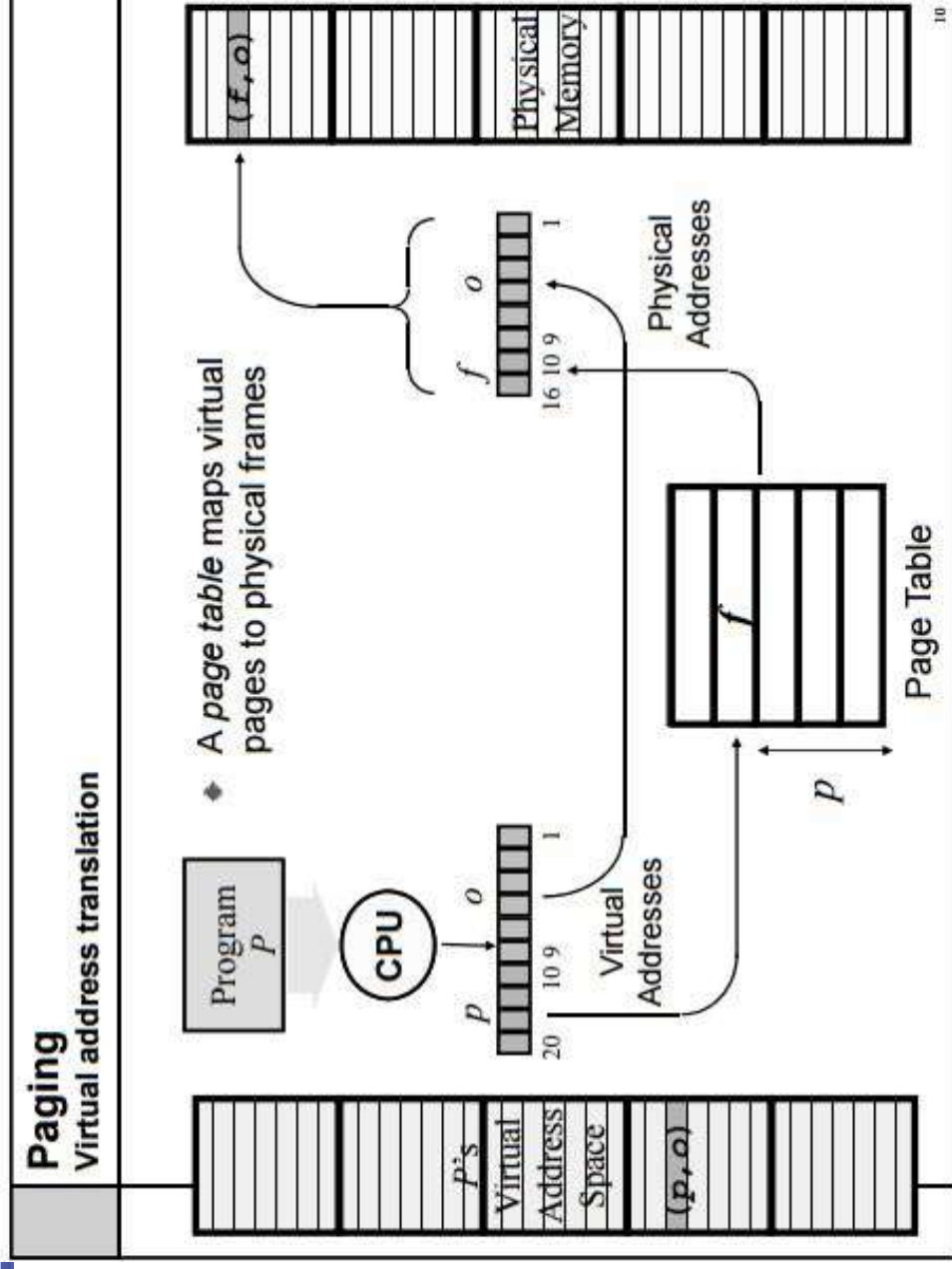
TLB (Translation Lookaside Buffer)

- The **TLB** is a **small, fast cache** inside the CPU.
- It stores **recent translations** from virtual addresses to physical addresses (from the page table).
- Why?
 - **Page table lookups are slower.**
 - **TLB reduces memory access latency.**
- If the needed translation is **in the TLB (hit)** → fast access.
- If **not in TLB (miss)** → access the page table (slower), then update the TLB.

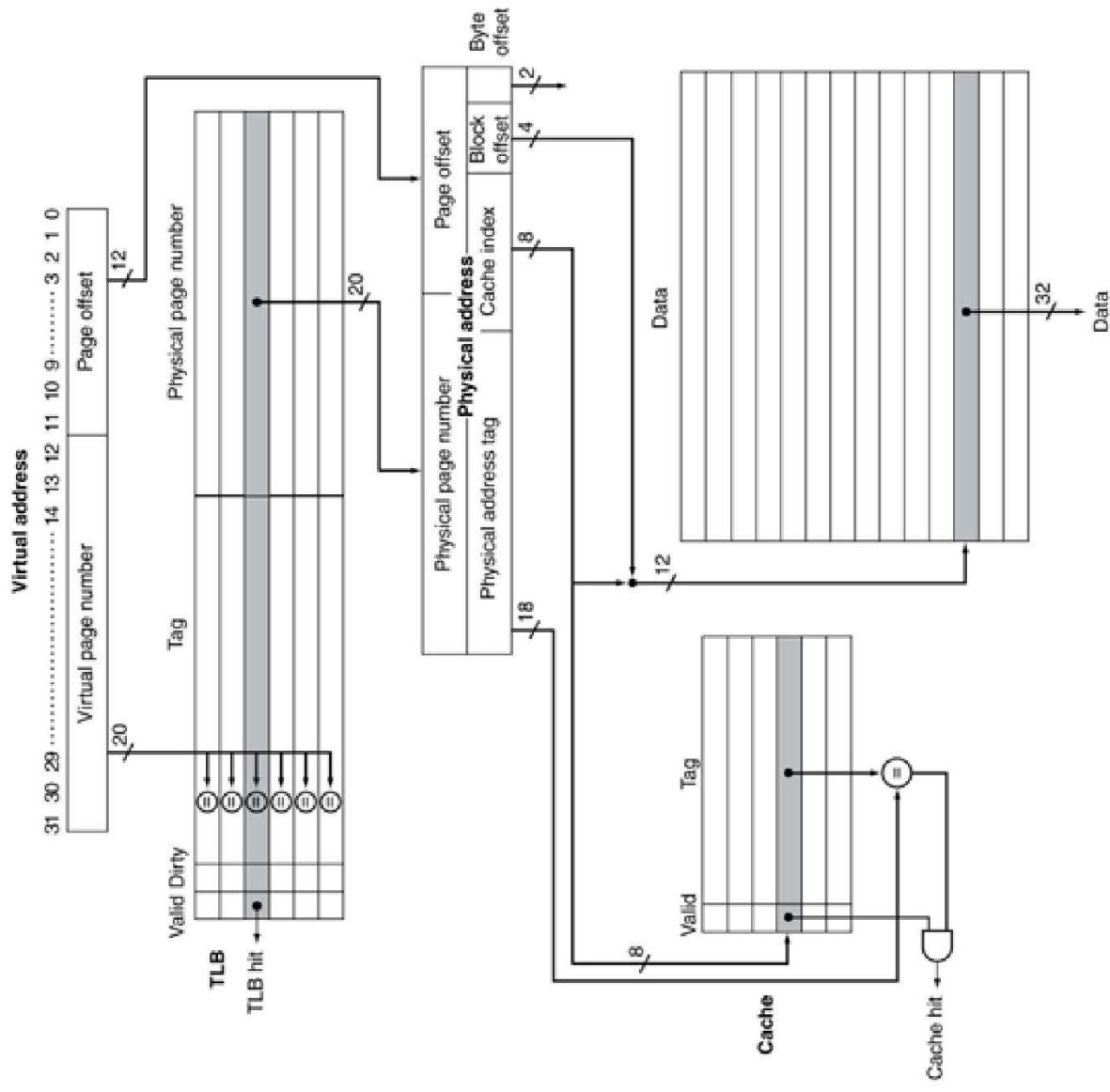
TLB (Translation Lookaside Buffer)

- When the CPU generates a **virtual address**, it needs to be **translated** into a **physical address** before accessing cache or main memory. This translation is done using the **TLB**.
- The **TLB is consulted before the L1 cache**, TLB is not in between L1 and main memory.
- It helps **translate virtual addresses to physical addresses**.
- Once the **physical address** is available, the **caches (L1, L2, etc.)** and eventually **main memory** can be accessed.

Conversion

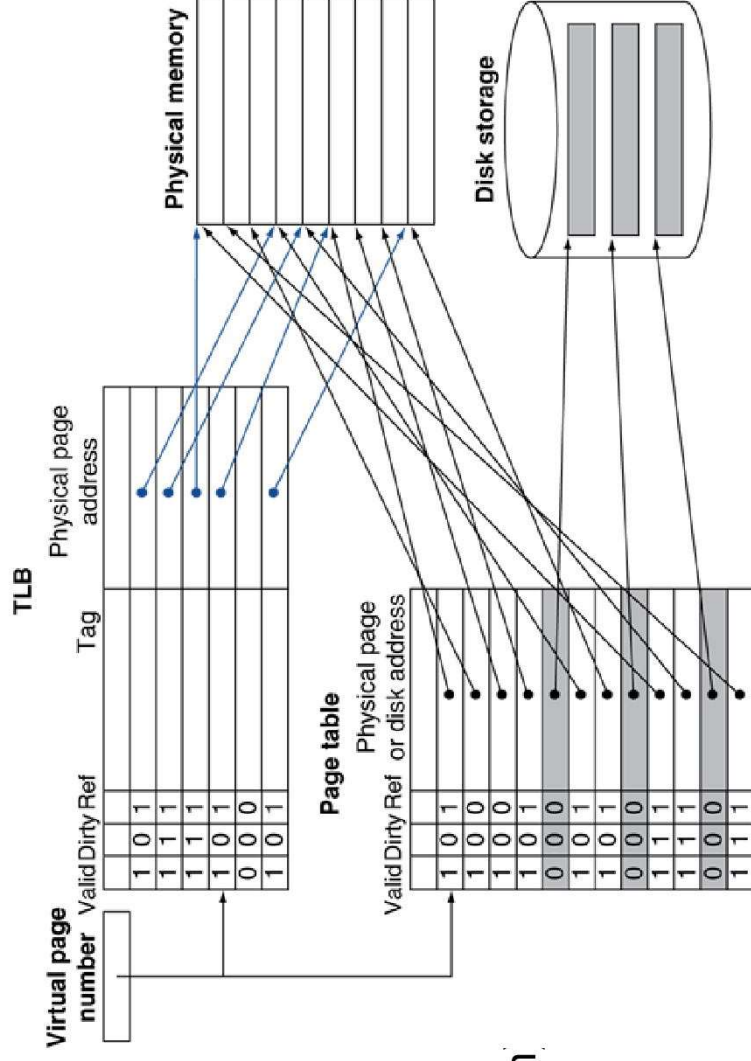


TLB and cache organization



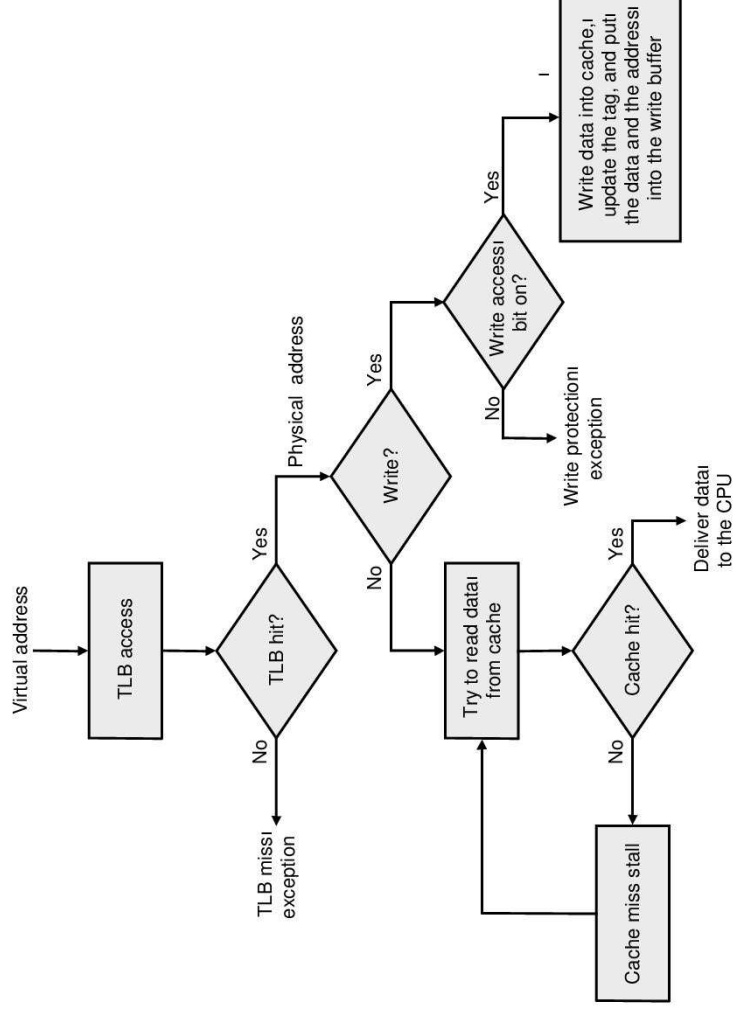
Optimizing address translation (TLB)

- Avoid two memory accesses for each read or write
- Rely on locality of reference to the page table
- Translation Lookaside Buffer (TLB) - cache holding page table mappings
- TLB tag: higher bites of the virtual address
- Valid bit, reference bit, dirty bit



- TLB size: 32-4096 entries
- Block size: 1-2 page table entries
- Hit time: 0.5-1 clock cycle
- Miss penalty: 10-30 clock cycles
- Miss rate: 0.01% - 1%
- Associativity: various (e.g. random)

DECStation 3100: processing read or write



Handling TLB misses and page faults

TLB miss (handled in software or hardware):

- The page is in memory, but its physical address is missing. A new TLB entry must be created
- The page is not in memory and the control is transferred to the operating system to deal with a page fault

Page fault (handled by the operating system):

- Causing exception (interrupt): using EPC and Cause register
- Instruction page fault:
 - Store the state of the process
 - Look up the page table to find the disk address of the referenced page
 - Choose a physical page to replace
 - Start a read from disk for the referenced page
 - Execute another process until the read completes
 - Restart the instruction which caused the fault
- Data access page fault:
 - Occurs in the middle of an instruction
 - MIPS instructions are restartable: prevent the instruction from completing and restart it from the beginning
 - More complex machines: interrupting instructions (saving the state of CPU)

Overall operation of a memory hierarchy

Cache	TLB	Page table	Possible? If so under what circumstance
miss	hit	hit	Possible, but never detected (page table is not checked after a TLB hit).
hit	miss	hit	TLB misses, but entry found in page table. After retry, found in cache.
miss	miss	hit	TLB misses, but entry found in page table. After retry, miss in cache.
miss	miss	miss	TLB misses and is followed by page fault. After retry, miss in cache.
miss	hit	miss	Impossible: cannot have translation in TLB if page is not in memory.
hit	hit	miss	Impossible: cannot have translation in TLB if page is not in memory.
hit	miss	miss	Impossible: data cannot be in cache if page is not present in memory.